Les API

L'API Calendrier

Table des matières

1. Présentation 1
2. Les classes techniques spécifiques aux API 3
   2.1. La classe ReponseHTTP 3
   2.2. La classe RequeteHTTP 4
   2.3. La clase Validation 5
3. Les classes métiers 6
4. La classe CalendrierSql 7
5. La classe CalendrierService 8
6. La classe CalendrierControleur 9
7. Tableau des requêtes valides de l'API 10
8. Mise en place d'un système minimaliste de routage 10
9. Le fichier index.php 11
   9.1. Analyse de la requête reçue 11
   9.2. Programmation de la méthode GET 12
   9.3. Mise en place de la procédure d'authentification dans le fichier index.php 12
   9.4. Mise en place des méthodes DELETE, POST et PATCH 13
10. Réalisation de la documentation 14
11. Mise en place d'une application cliente utilisant l'API pour gérer la table calendrier 15
   11.1. La classe métier ConsultationCalendrier 15
   11.1. La classe métier GestionCalendrier 15
   11.2. La consultation des épreuves inscrites au calendrier 16
   11.3. La gestion des épreuves 18
   11.4. L'ajout 19
   11.5. La mise à jour 20

Guy Verghote

Les API : L'API Calendrier

Objectif de l'activité professionnelle

A la fin de cette activité, vous serez capable :

De créer une API permettant de réaliser l'ensemble des opérations CRUD sur une table MySQL De mettre en place une application utilisant cette API pour gérer une table MySQL

1. Présentation

Vous êtes chargé de développer une API permettant de gérer le calendrier des courses organisées dans le département de la Somme.
L'API sera hébergée sur le Virtual Host nommé apicalendrier, associé au répertoire j:/virtualHostSlam/apicalendrier.
La version initiale de l'API est disponible sur GitHub, dans le dépôt nommé apicalendrier.

Ce calendrier est mémorisé dans la table calendrier(id, date, ville, nom, distances, challenge, label, organisateur, email)

La colonne id représente le numéro national de la course, attribué par la F.F.A (Fédération Française d'Athlétisme)
La colonne distances indique la ou les distances proposées
La colonne challenge indique si la course est inscrite (true/false) au challenge Courses 80
La colonne label indique si la course possède un label FFA ce qui lui permet d'être qualificative pour les championnats de France. 4 valeurs sont acceptées : 'Pas de label', 'Bronze', 'Argent' et 'Or'

Le répertoire sql contient les scripts permettant la création :
- de la base api_calendrier et de la table calendrier,
- de l'utilisateur utilisé par l'API,
- des déclencheurs sur la table calendrier
- des enregistrements pour l'année 2027

L'API doit permettre de réaliser les opérations CRUD.

Le fichier index.php sert de contrôleur principal, en dirigeant les requêtes vers l'opération appropriée.

Le choix de l'opération (consultation, ajout, modification, suppression) s'effectue en fonction de la méthode utilisée (GET, POST, PATCH, DELETE) puisque l'url reste la même : http://apicalendrier

Le fichier .htaccess doit rediriger toutes les requêtes HTTP vers index.php, permettant ainsi un routage centralisé de l'API

Voici la structure de la réponse JSON pour les opérations de consultation

```
{ "error" : message d'erreur}  // en cas d'erreur (peu probable)
```

```
{ "data": [ { "id": , …}, {…} …]}
```

Voici la structure de la réponse JSON pour un ajout ou une modification en cas de réussite

```
{
   "data": {
      "id": "123456",
      …
      "nom": "MARATHON DES SAINT",
      "date": "2026-05-01",
      "distances": "42 km"
   }
}
```

Ces réponses doivent être retournées avec un code HTTP 200 (ou 201 pour les créations) et l'en-tête Content-Type: application/json.

Voici la structure des réponses JSON pour un ajout ou une modification en cas d'échec

```
{ "error": {
      "id": "Le numéro de l'épreuve doit être composé de 6 chiffres",
      "nom": "Le nom ne doit contenir que des lettres, chiffres, espaces, apostrophes, virgules et tirets",
      "ville": "La ville est obligatoire",
      "date": "La date ne peut pas être antérieure à aujourd'hui"
   }
}
```

Pour la suppression, le code http vaux 204

```
{ "data": "Épreuve supprimée"}
```

```
{ "error" : {
    "global" : "id inexistant"}
}
```

L'API rend obligatoire l'utilisation du format JSON pour le transport de données. Il ne sera donc pas possible de tester l'API sous Insomnia en utilisant 'Form data' ou 'Form Url Encoded'

2. Les classes techniques spécifiques aux API

Dans une architecture REST, la communication entre le client et le serveur repose sur des requêtes HTTP standardisées. Les réponses sont généralement fournies en JSON, ce qui facilite l'intégration avec différents types d'interfaces et renforce la séparation des couches applicatives.

Nous avons donc besoin de classes techniques qui ne possède pas de dépendance forte vers l'interface client (le navigateur) comme c'est le cas actuellement de nos classes Erreur, Database, Select etc.

Toutes ces classes ne sont donc pas utilisables dans une API.

Une version adaptée de la classe Database est donc utilisée dans laquelle la gestion des erreurs est réalisée par des appels de la méthode **ReponseHTTP**::**sendError**()

2.1. La classe ReponseHTTP

Elle centralise la gestion des réponses envoyées par l'API au client.
Elle a pour objectif d'uniformiser le format des réponses JSON et de simplifier le traitement des erreurs dans tous les contrôleurs.

La méthode **send**($reponse, $status= 200) :

- ✓ envoie une réponse Json "normale",
- ✓ définit le code http par défaut sur 200
- ✓ stoppe l'exécution du script immédiatement après la réponse.

La méthode **sendError**(string $message, int $code = 400, string $cle = 'global'):

- ✓ envoie une réponse Json,
- ✓ définit le code http par défaut sur 400
- ✓ stoppe l'exécution du script immédiatement après la réponse.

La méthode **sendCatchError**(string $message) gère les exceptions et erreurs imprévues (erreur technique, exception PHP, etc.).

Si le message contient le caractère ~, c'est considéré comme une erreur maîtrisée provenant d'un trigger. Dans ce cas

- ✓ Le texte après ~ est affiché directement.
- ✓ Le code HTTP reste 200.

Sinon, c'est une erreur interne :

- ✓ Le code HTTP devient 400 (ou 500 selon ta convention).
- ✓ Le message technique est enregistré via Journal::enregistrer().
- ✓ Le client reçoit un message neutre et user-friendly :

Dans tous les cas

- ✓ stoppe l'exécution du script immédiatement après la réponse.

2.2. La classe RequeteHTTP

Cette classe sert à **récupérer les données envoyées au serveur** par un client (navigateur, JavaScript fetch, ou outil comme Postman).

Elle agit comme une **interface unifiée** pour lire les données d'une requête HTTP, quel que soit :

- ✓ le **verbe HTTP** utilisé (POST, PATCH, PUT…)

Elle impose le format JSON pour envoyer les données

PHP remplit automatiquement la superglobale $_POST quand les données proviennent d'un formulaire HTML ou d'un envoi avec l'entête Content-Type: application/x-www-form-urlencoded ou Content-Type: multipart/form-data

Quand les données ne sont **pas envoyées sous forme de formulaire**, mais en **JSON pur**, ou dans des verbes HTTP où $_POST n'est pas rempli, il faut lire le corps brut d'une requête http (php://input)

La méthode getData() vérifie que l'entête 'CONTENT_TYPE' contient bien 'application/json' puis récupère les données brutes en JSON via la fonction file_get_contents('php://input').

Ces données brutes sont ensuite décodées via la fonction json_encode pour obtenir un tableau associatif.

Rappel : Lorsqu'on envoie des données depuis un formulaire HTML ou un client AJAX, il existe deux grands modes :

En utilisant un formulaire classique (application/x-www-form-urlencoded ou multipart/form-data)

```
<form method="POST" action="...">
```

Les données sont automatiquement parsées et stockées dans le tableau superglobal $_POST **mais uniquement avec la méthode POST**.

En envoyant les données dans le corps de la requête dans le format JSON

```
fetch("api.php", { method:"POST", body: JSON.stringify({nom:"…"}) })
.
```

Dans ce cas, PHP ne remplit pas $_POST, il faut lire le corps brut de la requête via **file_get_contents('php://input')** et le décoder avec json_decode().

2.3. La clase Validation

La classe Validation est une classe qui permet de valider n'importe quel tableau de données (provenant d'un formulaire ou d'une API) selon un ensemble de règles spécifiques à chaque donnée.

Elle est **statique**, donc utilisable partout sans instanciation
Elle ne dépend d'aucune entité spécifique, ce qui la rend réutilisable dans différents services ou modules.

Elle centralise les contrôles sur les données et retourne les données validées (mise en forme) et un tableau des erreurs rencontrées.

Elle permet de **séparer la logique de validation** de la logique métier.
Elle facilite la **maintenance et l'évolution des règles** sans toucher aux services.

La méthode valider() prend 2 paramètres :

- ✓ $data : les données à vérifier
- ✓ $lesRegles : un tableau décrivant les contraintes pour chaque champ (type, longueur, valeurs possibles...).

Elle renvoie un tableau à deux éléments :

- ✓ $data → les données corrigées/nettoyées (chaînes trimées, dates formatées...),
- ✓ $erreurs → un tableau associatif des erreurs détectées (clé = nom du champ, valeur = message d'erreur).

Elle est utilisée dans la classe CalendrierService

La méthode valider() réalise les traitements suivants :
Suppression des espaces parasites dans les chaînes de caractères.

Pour chaque champ, on vérifie :
S'il est fourni ou absent.
S'il est obligatoire, facultatif, ou a une valeur par défaut.
Son type pour appliquer les validations spécifiques (string, date, enum…).

Les erreurs sont stockées dans un tableau associatif dont la clé est le nom du champ

La méthode renvoie toujours les données nettoyées et le tableau d'erreurs, ce qui permet au service ou au contrôleur de décider quoi faire ensuite (ajouter, modifier, afficher un message, etc.).

3. Les classes métiers

L'API est construite selon une architecture en couches.

Couche	Classe	Rôle	Interagit avec
Contrôleur	CalendrierControleur	Récupère les requêtes HTTP, appelle les services appropriés, et retourne une réponse JSON.	CalendrierService
Service (Métier)	CalendrierService	Coordonne les opérations métier : applique les règles métier, effectue les validations, appelle le DAO si les données sont valides.	Validation CalendrierSql
Données (DAO)	CalendrierSql	Gère les interactions directes avec la base de données : exécute les requêtes SQL pour le CRUD.	CalendrierService

Les avantages sont nombreux

- **Lisibilité** : chaque classe a un rôle bien défini
- **Testabilité** : chaque couche peut être testée indépendamment
- **Réutilisabilité** : la couche Service peut être utilisée ailleurs (ex. ligne de commande, tests, autres interfaces)
- **Robustesse** : structure claire, facilement évolutive
- **Sécurité** : les règles métier sont centralisées, évitant les contournements

Schéma simplifié des interactions dans le cadre d'un ajout

Client HTTP (POST)

↓ *uri demandée avec la méthode (POST) et les données envoyées (php://input ou $_POST)*

Controleur index.php : analyse l'Uri et la méthode pour déterminer la méthode à appeler

↓ *CalendrierControleur::ajouter();*

CalendrierControleur

*Récupère les données : $data = **RequeteHTTP**::getData();*

Instancie un objet CalendrierService : $service = new CalendrierService();

↓ *Exécute sa méthode ajouter : return $service->ajouter($data);*

CalendrierService

*Contrôle les données: [$data, $erreurs] = **Validation**::valider($data, $this->lesRegles);*

↓ *Exécute la méthode ajouter de l'objet : $ok = $this->calendrierSql->ajouter($data);*

CalendrierSql

↓ *Retourne le résultat de la méthode ajouter : true/false*

CalendrierService

Retourne une réponse dans un tableau associatif

↓ *Ce tableau comporte soit une clé 'data' soit une clé 'error'*

CalendrierControleur

Envoie la réponse dans le format json

4. La classe CalendrierSql

Contrairement à nos développements actuels, la classe CalendrierSQL n'est pas statique. Un objet de cette classe contient un objet PDO assurant la connexion et l'interaction avec la base de données.

	Classe statique	Classe non statique
Avantage	Simplicité d'usage Performance : pas d'objet à créer	Injection de dépendance (Db) Testabilité Héritage
Inconvénient	Couplage fort (classe Database) Pas de test unitaire possible Pas d'héritage donc duplication de code	Complexité plus élevée Performance Pas forcément utile sur des projets simples

La classe CalendrierSql Calendrier permet de réaliser l'ensemble des opérations de gestion sur les données de la table calendrier, elle comporte les méthodes suivantes :

- getAll() : retourne l'ensemble des enregistrements
- getById($id) : retourne l'enregistrement dont l'id est passé en paramètre
- ajouter($data) : création d'un enregistrement
- modifier($idActuel, $data) : modification d'un enregistrement
- supprimer($id) : supprimer l'enregistrement dont l'id est passé en paramètre

Aucune gestion explicite des exceptions n'est implémentée dans cette classe (pas de try/catch).
Toute erreur générée par le moteur PDO ou par PHP (ex. PDOException, TypeError) est automatiquement remontée jusqu'au niveau du contrôleur (CalendrierControleur), qui est responsable de son traitement ou de la transformation en réponse API.

1. Compléter les méthodes de la classe CalendrierSql

5. La classe CalendrierService

Elle est chargée de vérifier les données transmises et d'appeler les méthodes de la classe CalendrierSql pour réaliser l'opération au niveau de la base de données.

Elle comporte deux propriétés :
Un objet de la classe CalendrierSql afin de réaliser l'opération dans la base de données
Un tableau $lesRegles contenant pour chaque champs la liste des règles à respecter

Elle utilise la méthode valider(array $data, array $lesRegles): array de la classe technique Validation pour réaliser les vérifications.
Elle laisse remonter les erreurs inattendues vers la classe CalendrierControleur

Chaque méthode retourne en cas de succès un tableau associatif contenant la clé 'data'

En cas d'échec (données invalides, épreuve introuvable, conflit), elles **lancent une exception** (throw new ErreurHTTP(...)). C'est ensuite la classe CalendrierControleur qui attrape cette exception et transforme proprement les erreurs au format JSON structuré avec la clé 'error'.

La clé 'error' contient un tableau associatif ou chaque clé correspond à un champ de la table ou à 'global'.

La structure du tableau $lesRegles est la suivante :

```
[
    'id' => [
            'required' => true,
            'type' => 'string',
            'regex' => '/^[0-9]{6}$/',
            'messages' => ['regex' => "Le numéro d'épreuve doit être composé de 6 chiffres"]
    ],
    'nom' => [
            'required' => true,
            'type' => 'string',
            'min' => 3,
            'max' => 150,
            'regex' => "/^(?![\s\-',])[\p{L}0-9'\s\-]+(?<![\s\-',])$/u",
            'messages' => ['regex' => "Le nom ne doit contenir que lettres, chiffres, espaces,
apostrophes, virgules et tirets"]
    ],
    …
]
```

Pour chaque champ, on définit la valeur des règles 'require', 'type', 'regex', 'min', 'max' etc.
La propriété 'messages' permet d'associer à chaque règle un message d'erreur personnalisé.

2. Etudier la classe CalendrierService et indiquer la signification de l'expression régulière : /^(?![\s\-',])[\p{L}0-9'\s\-]+(?<![\s\-',])$/u

6. La classe CalendrierControleur

Elle récupère les requêtes HTTP, appelle les services appropriés, et retourne une réponse JSON.
Toutes les méthodes doivent être encapsulées dans un bloc try catch, pour traiter toutes les erreurs qui remontent.
Deux classes d'erreurs sont utilisées :

ErreurHTTP : Intercepte les erreurs métiers ou de validation prévues (ex : 400, 404, 409, 415, 422) et transmet les codes de statut HTTP précis ainsi que les messages d'erreur associés via ReponseHTTP::sendErreurs().

Throwable : Attrape toutes les autres exceptions techniques non prévues (ex : plantage de connexion à la base de données) pour renvoyer une erreur serveur propre (code 500) via ReponseHTTP::sendCatchError().

Le modèle est le suivant :

```
try {
    // instanciation du service
    $service = new …();
    // Appel de la méthode
    $reponse = $service->…;
    // Envoi de la réponse HTTP
    ReponseHTTP::send($reponse);
} catch (ErreurHTTP $e) {
    ReponseHTTP::sendErreurs($e->getErreurs(), $e->getStatut());
} catch (Throwable $e) {
    ReponseHTTP::sendCatchError($e->getMessage());
}
```

5 méthodes sont à compléter :

getAll() : Point d'entrée pour les requêtes GET sans identifiant.
Appelle la méthode getAll() du service pour récupérer l'ensemble des épreuves et les renvoie au client au format JSON.

getById(string $id) : Point d'entrée pour récupérer le détail d'une épreuve spécifique via son identifiant FFA (GET /id).
Transmet l'ID au service, gère le cas où l'épreuve n'existe pas (générant une erreur 404 gérée par le service) et renvoie l'épreuve trouvée.

ajouter() : Point d'entrée pour créer une nouvelle épreuve (POST /).

1. Récupère et valide le format des données entrantes (exige obligatoirement du application/json).
2. Vérifie que le corps de la requête n'est pas vide (sinon renvoie une erreur 400).
3. Appelle le service pour l'insertion et renvoie une réponse de succès avec le code HTTP **201 (Created)**.

modifier(string $id) Point d'entrée pour la modification d'une épreuve existante (PATCH /id).

1. Récupère les données au format JSON.
2. Vérifie la présence des données.
3. Transmet l'ID et les données au service pour fusion et mise à jour, puis renvoie l'épreuve modifiée avec un code HTTP **200**.

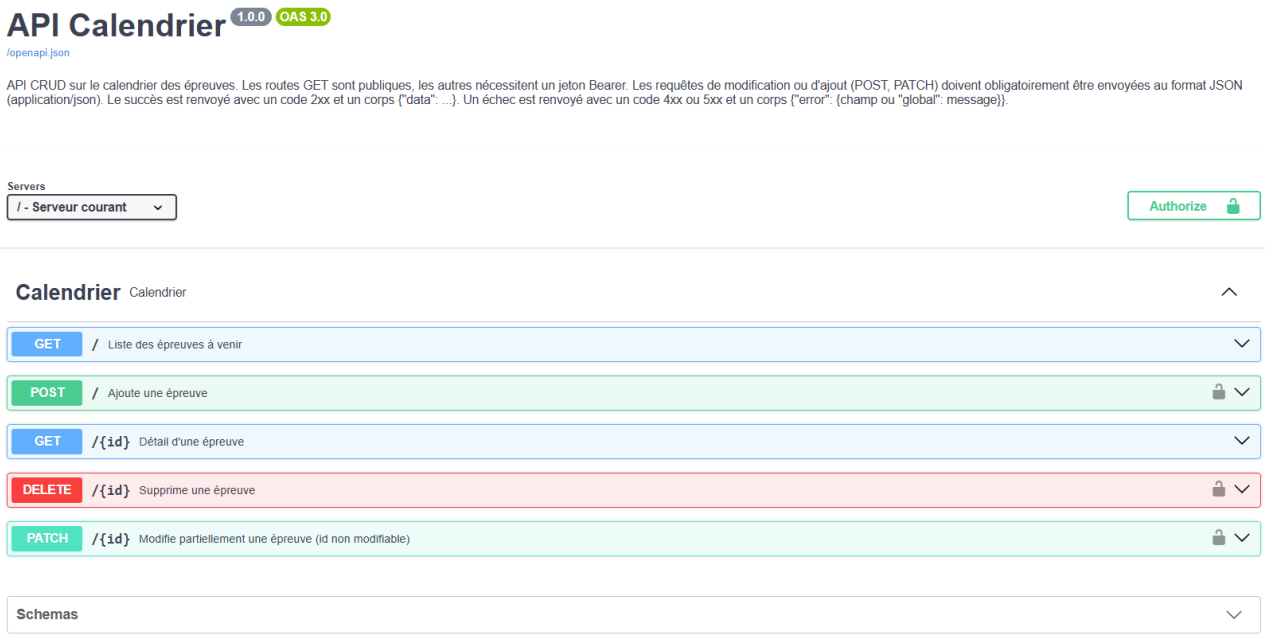
supprimer(string $id) Point d'entrée pour supprimer une épreuve (DELETE /id).
Demande au service de supprimer l'épreuve ciblée par son identifiant FFA et renvoie un message de confirmation de suppression.

3. Compléter les méthodes de la classe CalendrierControleur

7. Tableau des requêtes valides de l'API

Méthode	URL	Action
GET	http://apicalendrier/	Retourne le calendrier complet (toutes les épreuves)
GET	http://apicalendrier/{id}	Retourne les informations détaillées d'une épreuve
POST	http://apicalendrier/	Ajoute une nouvelle épreuve dans le calendrier
DELETE	http://apicalendrier/{id}	Supprime une épreuve identifiée par son ID
PATCH	http://apicalendrier/{id}	Modifie les informations d'une épreuve existante

La documentation de l'API est consultable depuis l'url http://apicalendrier/docs/



8. Mise en place d'un système minimaliste de routage

Toutes les requêtes doivent être redirigées vers le script index.php sauf si le fichier ou le dossier demandé existe bien.

Pour cela il suffit de mettre en place une redirection dans le fichier .htaccess à la racine de l'API

Aide technique : Le module de réécriture d'Apache (mod_rewrite)

Pour configurer un Front-Controller tout en préservant l'accès aux fichiers existants, on utilise généralement les directives suivantes :

- Activer le moteur : RewriteEngine On
- Vérifier si une cible existe physiquement (fichier -f ou dossier -d) : RewriteCond %{REQUEST_FILENAME} ...
- Appliquer une règle de redirection globale vers un script unique : RewriteRule expReg fichier

4. Compléter le fichier .htaccess pour réaliser cette redirection

9. Le fichier index.php

9.1. Analyse de la requête reçue

Il faut commencer par analyser la requête afin de vérifier sa validité et éventuellement d'en extraire le paramètre {id}

Pour récupérer les différents segments constituant de l'URI :

$_SERVER['REQUEST_URI'] : contient l'URI de la requête actuelle, y compris le chemin et les paramètres de la requête.

Exemple avec l'url http://apicalendrier/calendrier/1
$_SERVER['REQUEST_URI'] retourne /calendrier/1

parse_url($_SERVER['REQUEST_URI'], PHP_URL_PATH) : analyse l'URI et extrait uniquement le chemin (sans les paramètres de la requête). Dans notre cas cela ne change rien puisque la requête ne comporte pas de paramètre

trim($uri, '/') : supprime les barres obliques (/) au début et à la fin de la chaîne $uri.

explode('/', trim($uri, '/')) : divise la chaîne $uri en segments en utilisant la barre oblique (/) comme délimiteur.

Si l'URL contient plus d'un segment, if faut renvoyer une erreur « chemin invalide » ; si elle contient exactement un segment, ce segment doit être un nombre, qui sera alors utilisé comme identifiant $id.

Une fois que l'on sait si on cible la collection entière ou une épreuve spécifique (/{id} donc $id est renseigné), il faut s'assurer que la méthode HTTP utilisée par le client a le droit d'être appelée sur cette ressource.

Sur la racine (/), on autorise la consultation globale (GET) et la création (POST).
Sur une épreuve ciblée (/{id}), on autorise la consultation d'un détail (GET), la modification partielle (PATCH) et la suppression (DELETE), mais pas la création.

$_SERVER['REQUEST_METHOD'] indique si le client a envoyé un GET, POST, PUT, etc.
strtoupper(...) permet de s'assurer que la méthode est bien en majuscules pour éviter les erreurs de comparaison (ex: get vs GET).
La fonction in_array($method, $methodesAutorisees, true) vérifie de manière stricte (true) si la méthode demandée fait partie de celles autorisées.

Si la méthode et la ressource sont incompatible, le protocole HTTP impose deux choses de bonnes pratiques :
Envoyer un en-tête Allow qui indique au client quelles sont les méthodes *réellement* permises : header('Allow: ' . implode(', ', $methodesAutorisees));
Renvoyer une erreur propre avec le **code HTTP 405** (Méthode non autorisée) via ReponseHTTP::sendError(...).

5. En suivant les explications ci-dessus, complétez le code de l'étape 1 du fichier index.php.

6. Vérifications sous Insomnia en testant les URL ci-dessous (en méthode GET pour commencer) et vérifiez que vous obtenez bien le comportement attendu

URL testée	**Comportement / Réponse attendue**	**Code HTTP**
http://apicalendrier/	Racine valide (liste des épreuves - géré à l'étape suivante)	200 ou autre (pas de 404)
http://apicalendrier/325932	Un segment valide (6 chiffres) → l'identifiant $id est bien extrait	200 ou 404 (si l'épreuve n'est pas encore en BDD, mais le routage de l'URI est validé)
http://apicalendrier/123	L'identifiant n'a pas 6 chiffres → Erreur "Ressource introuvable"	404
http://apicalendrier/abc	L'identifiant contient des lettres → Erreur "Ressource introuvable"	404
http://apicalendrier/calendrier/325932	Trop de segments (plus de 1) → Erreur "Ressource introuvable"	404

9.2. Programmation de la méthode GET

La méthode GET est utilisée pour lister les épreuves du calendrier ou pour renvoyer les informations sur une épreuve dont l'identifiant est passé en paramètre

http://apicalendrier	http://apicalendrier/**319254**

Le fichier index.php doit tester la méthode utilisée puis la ressource afin de pouvoir lancer la bonne méthode de la classe CalendrierControleur réalisant l'action demandée : getById ou getAll.

7. Compléter le code de l'étape 2 du fichier index.php et vérifier le fonctionnement de la méthode GET sous Insomnia.

9.3. Mise en place de la procédure d'authentification dans le fichier index.php

L'ajout, la suppression et la modification nécessite une procédure d'authentification.
Afin de mettre en œuvre rapidement et simplement un contrôle d'authentification nous utilisons un 'bearer token' (jeton porteur) qui ne nécessite pas de signature cryptographique de chaque requête.

Le compromis est que toutes les demandes d'API doivent être effectuées via une connexion HTTPS, car la demande contient un jeton en texte brut qui pourrait être utilisé par n'importe qui s'il était intercepté.
Ce token est normalement mémorisé sur le serveur pour chaque utilisateur autorisé avec une durée de validité (comme sur GitHub). Ici nous allons utiliser pour nos tests un jeton prédéfini (puisque nous n'avons pas de table gérant les utilisateurs autorisés)

La valeur attendue du jeton est conservée coté serveur dans une variable d'environnement dans le fichier public/.htaccess

```
SetEnv CALENDRIER_TOKEN *****
```

Ce jeton est comparé au jeton transmis dans la requête du client pour autoriser l'opération demandée dans cette requête.

En PHP, la fonction getallheaders() retourne dans un tableau associatif les entêtes de la requête. On y trouve la clé 'Authorization'
Si cette clé n'existe pas il faudra retourner un message "Une authentification est requise" avec un code 401 en utilisant la méthode ReponseHTTP::*sendError*

Si la clé existe mais sa valeur ne correspond pas à celle attendue, il faudra retourner le message "Le jeton d'authentification n'est pas valide" avec un code 401.

Pour récupérer la valeur du token : trim(str_replace("Bearer", "", $headers['Authorization']));
Pour récupérer la valeur du token conservé sur le serveur : getenv('CALENDRIER_TOKEN');
hash_equals($tokenAttendu, $token) est une fonction de sécurité indispensable en PHP. Elle permet de comparer deux chaînes de caractères en temps constant, ce qui protège l'application contre les attaques informatiques par analyse temporelle (timing attacks)

8. Compléter le code de l'étape 3 du fichier index.php et vérifier ces contrôles sous Insomnia.

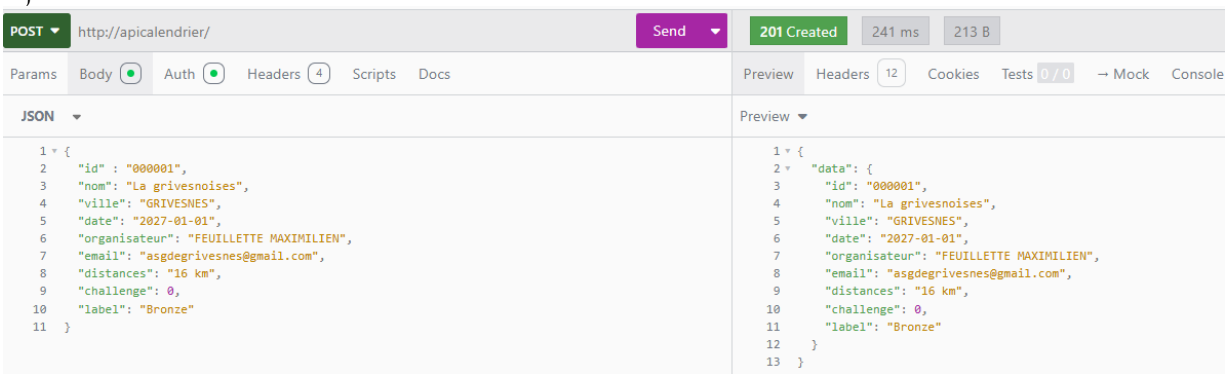
9.4. Mise en place des méthodes DELETE, POST et PATCH

Le routeur a réalisé l'ensemble des actions de vérification : il a filtré, validé l'URI, vérifié les méthodes, vérifié le token, et rejeté tout ce qui n'est pas valide (404, 405, 401).

Il lui reste maintenant à lancer la méthode de la classe CalendrierControleur qui correspond à la méthode utilisée par la requête : 'DELETE', 'POST, 'PATCH'

9. Compléter le code de l'étape 4 du fichier index.php et vérifier le fonctionnement des méthodes sous Insomnia.

Ajout



En cas de doublon lors d'un ajout (relancer la même requête)

```
{"error": {"id": "Ce numéro d'épreuve est déjà attribué"   }}
```

Si transmission via Form Data

```
{  "error": { "global": "Le format des données doit être obligatoirement application/json"}}
```

En cas d'erreur sur l'id

```
{ "error": { "global": "Cette épreuve n'existe pas" }}
```

En cas d'erreur sur les données

```
{
  "error": {
    "challenge": "Le champ challenge 2 doit valoir 0 ou 1",
    "email": "L'adresse email est invalide"
  }
}
```

10. Réalisation de la documentation

Dans une architecture API REST moderne, la documentation est indispensable pour structurer les échanges et permettre au front-end d'interagir proprement avec le serveur.

OpenAPI est le standard international de description d'API.
Swagger : C'est l'ensemble des outils (comme Swagger UI) qui permettent de lire cette spécification OpenAPI pour générer une interface graphique web interactive et élégante.
zircote/swagger-php est une bibliothèque PHP qui permet de générer automatiquement cette documentation directement à partir du code source, grâce à des commentaires annotés (ex: @OA\Post, @OA\Patch, @OA\JsonContent).

La démarche se résume en 4 étapes
Installation du paquet via Composer (zircote/swagger-php).

Annotation des contrôleurs : Ajout des blocs de commentaires structurés au-dessus des scripts (comme ajouter.php et modifier.php) pour décrire les routes, les paramètres (dont la structure des champs modifiés) et les réponses HTTP attendues.
Astuce : Connaître et rédiger toute la syntaxe détaillée de ces annotations peut s'avérer fastidieux. C'est typiquement une étape que l'IA peut prendre en charge pour vous en générant les blocs d'annotations à partir de votre code source.)

Génération du fichier de spécification (format JSON/YAML) via la ligne de commande du package.

```
vendor/bin/openapi src/ -o public/openapi.json
```

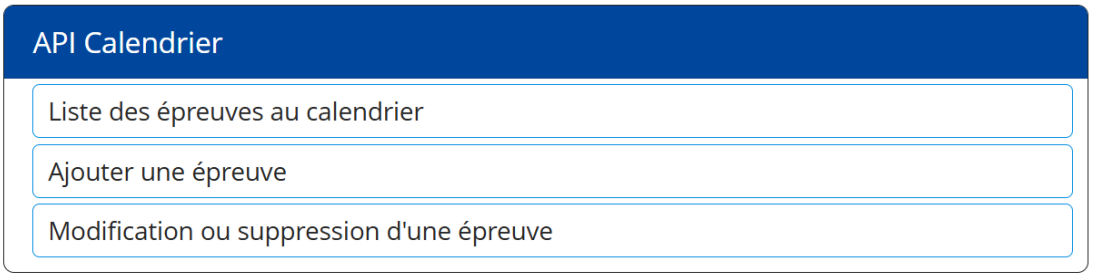
Création di fichier public.docs/index.html mettant en place l'interface graphique Swagger UI connectée au fichier généré.

```
<div id="swagger-ui"></div>
<script src="https://unpkg.com/swagger-ui-dist@5/swagger-ui-bundle.js"></script>
<script>
   SwaggerUIBundle({url: '/openapi.json', dom_id: '#swagger-ui'});
</script>
```

La documentation complète, est accessible dans la base de connaissances Obsidian : "Documentation d'API avec Swagger et zircote swagger-php".

11. Mise en place d'une application cliente utilisant l'API pour gérer la table calendrier

Notre projet API permet de tester notre API calendrier



11.1. La classe métier ConsultationCalendrier

Cette classe nous permet d'interroger l'API 'apicalendrier' en utilisant la méthode GET
Comme dans le cadre de l'utilisation de l'API Github, cette classe utilise le trait HttpClient., mais cette fois-ci sans nécessiter la mise en place d'une méthode spécifique.

10. Compléter la classe ConsultationCalendrier

11.1. La classe métier GestionCalendrier

Cette classe nous permet d'interroger l'API 'apicalendrier' en utilisant la méthode POST, PATCH et DELETE

Le token d'autorisation est initialisé dans le constructeur

```
public function __construct()  {
        $this->token = getenv('CALENDRIER_TOKEN') ?: '';
}
```

Ce token est conservé dans une variable d'environnement définie dans le fichier calendrier/.htaccess

La méthode executer() provenant du trait HttpClient est utilisée dans chaque méthode publique.

Rappel sur la méthode requête du trait HttpClient :.
Elle reçoit ici en paramètre :

- ✓ La méthode HTTP à utiliser (GET, POST, PUT, DELETE)
- ✓ Le endPoint à interroger : **partie relative à partir de la racine du serveur**
- ✓ Les données à envoyer avec la requête (pour POST/PUT). Null si aucune donnée.
- ✓ Le token d'autorisation.

Vous pouvez mettre en œuvre ici le principe des paramètres nommés pour vous permettre de fournir les paramètres dans l'ordre de son choix et d'indiquer seulement les paramètres nécessaires

11.Compléter le code des méthodes publiques de la classe GestionCalendrier

11.2. La consultation des épreuves inscrites au calendrier



Calendrier des épreuves de la Somme

ID	Nom & Ville	Date	Distances	Chal.	Label	Organisateur
325932	Course Monsboubert	09/01/2027	10 km et 3 km			Fournier Christian - christian.fournier77@sfr.fr
326563	Trail de Saint Fuscien	10/01/2027	8 km, 16 km et 24 km			Auc Athletisme - auc.athletisme@wanadoo.fr
328277	Trail du renard givré (Cottenchy)	30/01/2027	13 km et 28 km			Raid En Somme - raidinsomme.orga@gmail.com
329855	La CapSolid'Erre (Doullens)	31/01/2027	Non spécifiées			Adapmmac - capsoliderre@gmail.com
319254	Les foulées Méaultoises (Meaulte)	07/02/2027	5 km et 10 km			Pilot Dimitri - a.m.a.a.c@laposte.net
320283	Les quatre saisons d'hiver (Amiens)	14/02/2027	5 et 10 km			Amicale Du Val De Somme - contact@valdesomme.net
324104	La Flesselloise (Flesselles)	21/02/2027	10 km			Merelle Luc - luc.merelle@orange.fr
321132	Cottenchy trail aventure	07/03/2027	30 km et 13 km-mn			Molin Jean Paul - contact@vytajog.fr
327316	Course solidaire féminine (Camon)	07/03/2027	5 km			Evrard Jean Yves - jyevrard@laposte.net

Nous choisissons ici de récupérer les épreuves en réalisant un appel Ajax.

Le script index.js doit donc réaliser cet appel Ajax en appelant via la méthode GET l'url http://apicalendrier. La fonction afficher sera appelée en cas de réussite.

Nous perdons ici l'avantage de pouvoir obtenir directement depuis l'API la date dans le format français car l'API renvoie la date dans son format MySQL

La classe Javascript Date permet d'instancier un objet Date à partir d'une date au format MySQL et la méthode **toLocaleDateString('fr-FR')** retourne la date dans le format français

Pour éviter de répéter la même instruction pour accéder à chaque cellule par son attribut data 'champ' une fonction d'accès doit être utilisée

```
const champ = (nom) => ligne.querySelector(`[data-champ="${nom}"]`);
```

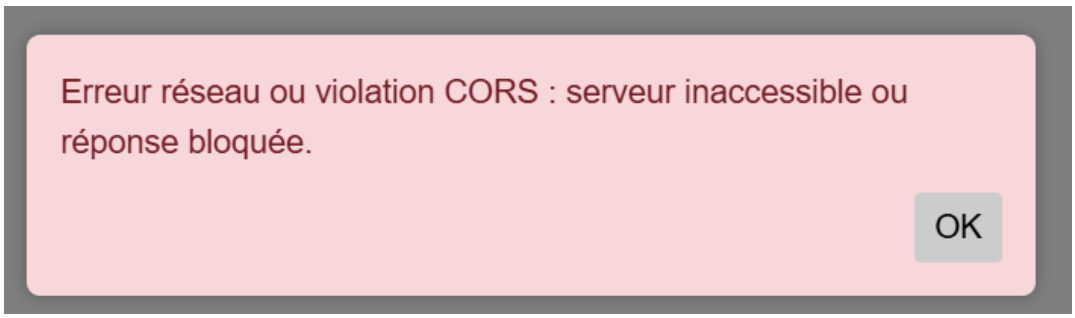
Elle permettra ainsi d'accéder aux cellules :

```
champ('nom').textContent = titre;
```

La balise <template> utilisée pour générer les lignes du tableau comporte au niveau de la colonne id une balise <a> permettant d'afficher dans un autre onglet les résultats de la course disponible sur le site de la FFA.
https://www.athle.fr/bases/liste.aspx?frmbase=resultats&frmmode=1&frmespace=0&frmcompetition=${epreuve.id}

12. Compléter le script consultationcalendrier/index.js

L'exécution de ce module affiche le message d'erreur suivant :



En effet, par sécurité, les navigateurs appliquent la politique de la même origine (Same-Origin Policy).

Cela veut dire que notre domaine http://api ne peut pas recevoir la réponse d'une requête envoyée vers un autre domaine (ici http://apicalendrier)

Cette restriction vise à protéger les utilisateurs contre des attaques du type vol de données (CSRF, XSS, etc.).

CORS (Cross-Origin Resource Sharing) est un mécanisme standard qui permet au serveur distant de dire explicitement au navigateur : "Oui, j'autorise tel site à faire des requêtes vers moi en utilisant les méthodes suivantes."

C'est donc le serveur qui accorde le droit, et le navigateur qui fait respecter la règle.

```
// Accès autorisé depuis le site http://api
header("Access-Control-Allow-Origin: http://api");
```

Si on souhaite autoriser tous les sites :

```
header("Access-Control-Allow-Origin: *");
```

Le mécanisme CORS natif n'accepte qu'une seule URL ou * dans l'en-tête Access-Control-Allow-Origin.
Si on souhaite autoriser plusieurs sites, Il faut vérifier dynamiquement l'origine de la requête ($_SERVER['HTTP_ORIGIN']) par rapport à une liste blanche de domaines autorisés :

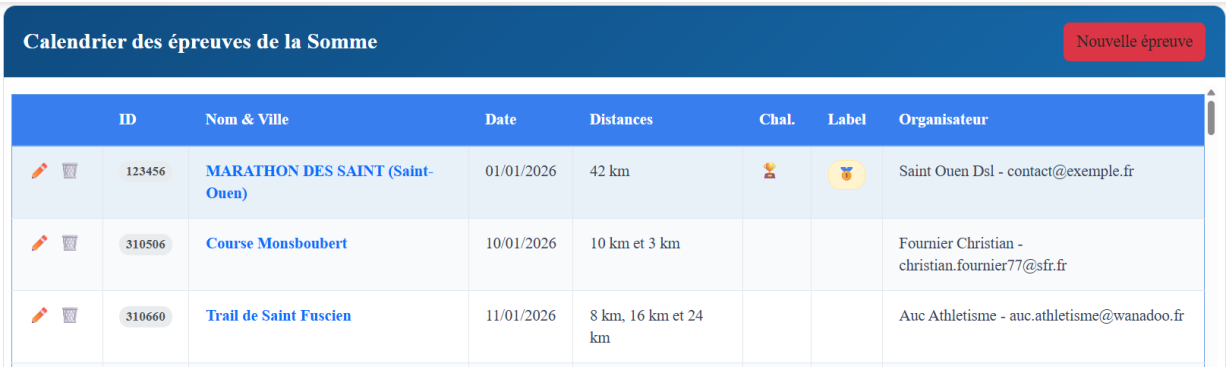
```
$domainesAutorises = [
    "http://api",
    "http://autre-site.fr"
];

if (isset($_SERVER['HTTP_ORIGIN'])
    && in_array($_SERVER['HTTP_ORIGIN'], $domainesAutorises)) {
        header("Access-Control-Allow-Origin: " . $_SERVER['HTTP_ORIGIN']);
        header("Access-Control-Allow-Methods: GET, POST, OPTIONS");
        header("Access-Control-Allow-Headers: Content-Type, Authorization");
}
```

13.Compléter le script index.php de l'API calendrier et vérifier que l'appel ajax est accepté.

11.3. La gestion des épreuves

L'interface de gestion permet de centraliser les opérations d'ajout, de modification et de suppression.



L'interface reprend le modèle de la consultation en y ajoutant une première colonne pour les actions de modification et de suppression.
Les données sont ici chargées par le contrôleur index.php ce qui s'avère plus rapide qu'un appel Ajax. De plus dans ce cas la politique de la même origine (Same-Origin Policy) ne s'applique pas

La politique de la même origine est une **mesure de sécurité exclusivement imposée par le navigateur web** lorsqu'un script exécuté côté client (JavaScript) tente d'effectuer une requête asynchrone (comme fetch ou XMLHttpRequest) vers un domaine, un protocole ou un port **différent** de celui qui a servi à charger la page web d'origine.
Dans le cas où les données sont chargées directement par le contrôleur PHP (index.php) lors du rendu initial de la page :

Il n'y a pas d'appel asynchrone (Ajax) : La page HTML et ses données sont générées directement côté serveur par le script PHP au moment de la requête HTTP initiale.
Le navigateur ne fait pas d'intermédiaire bloquant : C'est le serveur web lui-même qui assemble la structure et les données (via une requête SQL ou un appel interne). Pour le navigateur, il s'agit d'une simple page web classique (HTML statique ou rendu direct), il n'y a donc pas de "cross-origin request" JavaScript à analyser ou à bloquer.

14. Compléter le script gestioncalendrier/index.php

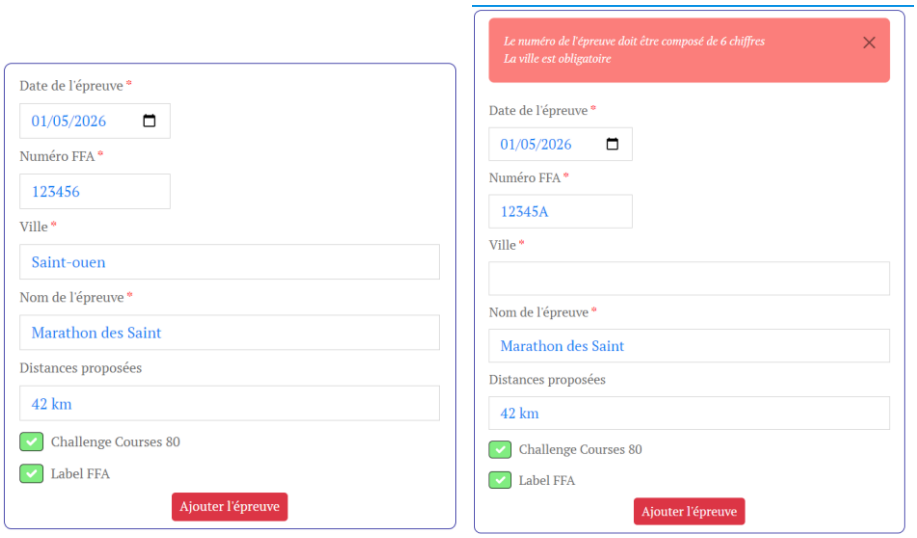
La seule opération réalisée directement par cette interface est la suppression via un appel Ajax au script gestioncalendrier/ajax/supprimer.php en transmettant l'id de l'épreuve

Ce script doit réaliser les opérations suivantes :

Vérification d'un appel AJAX par la méthode POST avec jeton
Récupération de l'identifiant de l'épreuve à supprimer
Création de l'objet métier pour gérer le calendrier
Appel de la méthode pour supprimer l'épreuve du calendrier
Si une erreur est survenue, envoyer l'erreur en réponse JSON
Si tout s'est bien passé, envoyer les données retournées par l'API (clé 'data') en réponse JSON

15. Compléter le script supprimer.php et réaliser les tests fonctionnels

11.4. L'ajout



Le script gestioncalendrier/ajout/ajax/ajouter.php doit réaliser les opérations suivantes :

Vérification d'un appel AJAX par la méthode POST avec jeton
Création de l'objet métier pour gérer le calendrier (GestionCalendrier)
Appel de la méthode ajouter() en lui transmettant les données brutes ($_POST).
Si une erreur est survenue, envoyer l'erreur en réponse JSON
Si tout s'est bien passé, envoyer les données retournées par l'API (clé 'data') en réponse JSON

On peut noter ici l'absence totale de validation ou de filtrage des données transmise ($_POST) dans ce contrôleur. Cela s'explique par l'utilisation rigoureuse de la couche service (CalendrierService et Validation::valider) :

Pas de validation métier dans le contrôleur : Ce n'est pas son rôle. Le contrôleur se contente de faire transiter les données brutes.

Le rôle du Contrôleur (ajouter.php) : C'est un simple chef d'orchestre (point d'entrée HTTP). Il vérifie la méthode HTTP, s'assure de l'authentification, appelle le service et renvoie la réponse au client.

Le rôle du Service (GestionCalendrier / CalendrierService) : Il contient toute l'intelligence et applique les règles de validation via son tableau $lesRegles et le validateur.

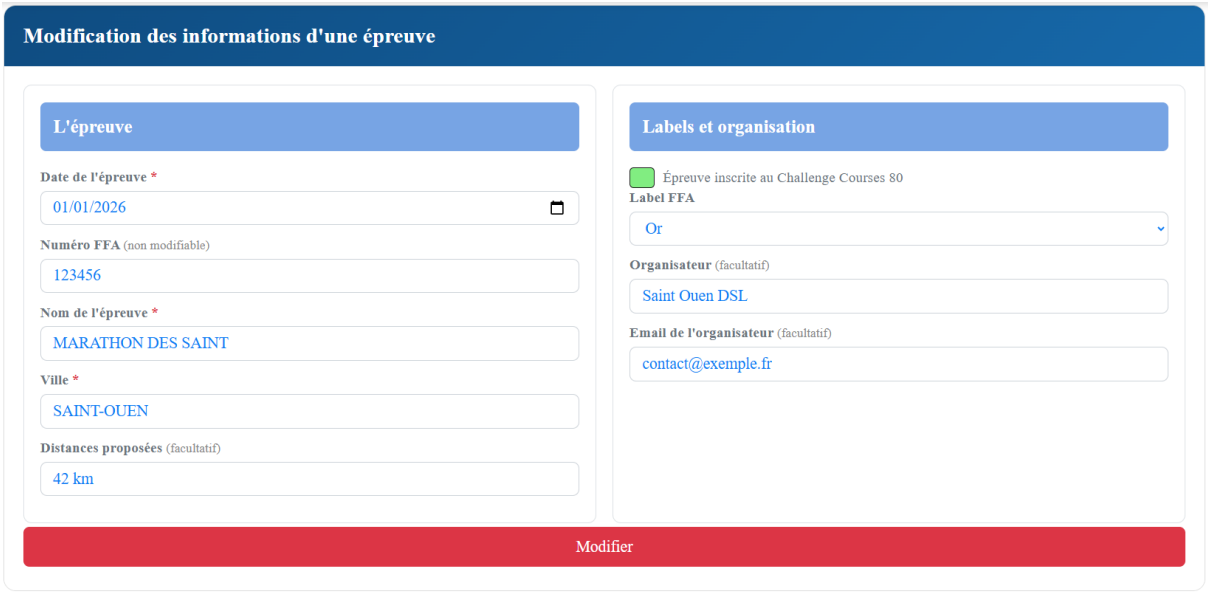
16.Compléter le script gestioncalendrier/ajout/ajax/ajouter.php

Le script **ajout/index.php** n'a aucune donnée à charger.

Le script **ajout/index.js** déclenche sur le clic du bouton 'Ajouter l'épreuve' un appel Ajax vers le script ajax/ajouter.php si les données sont valides. Les données facultatives (distances et email) ne sont transmises que si elles sont renseignées.

17.Réaliser les tests fonctionnels

11.5. La mise à jour



Le champ id ne doit pas être modifiable. Pour cela la balise input comporte l'attribut disabled

readonly : L'utilisateur ne peut pas modifier le texte, mais le champ reste focusable, sa valeur peut être sélectionnée/copiée, et **elle est envoyée au serveur** lors de la soumission du formulaire ou de l'appel AJAX. C'est souvent pratique pour qu'un utilisateur puisse copier le numéro FFA.
disabled : Le champ est grisé, totalement inactif, **mais sa valeur n'est pas envoyée** au serveur lors de la soumission native du formulaire (ce qui n'est pas un souci ici puisqu'on récupère l'ID directement en JavaScript via id.value pour l'injecter dans l'URL de l'API.

Le controleurt **ajax/modifier.php** doit réaliser les opérations suivantes :

Vérification d'un appel AJAX par la méthode POST avec jeton
Récupération de l'identifiant de l'épreuve à modifier et des valeurs à mettre à jour
Vérification que des valeurs ont été transmises pour la modification
Création de l'objet métier pour gérer le calendrier (GestionCalendrier)
Appel de la méthode modifier().
Si une erreur est survenue, envoyer l'erreur en réponse JSON
Si tout s'est bien passé, envoyer les données retournées par l'API (clé 'data') en réponse JSON

18.Compléter les scripts modifier.php.

Le fonction modifier() du script maj/index.js **maj.js** doit :
récupérer les données modifiées et s'il y en a,
réaliser l'appel ajax du script 'ajax/modifier.php'.
En cas de succès il faut :
mettre à jour la zone de liste car la date, le nom ou l'id de l'épreuve peuvent être modifiés
mettre à jour l'épreuve sélectionnée ce qui revient à mettre à jour le tableau lesEpreuves

19. Compléter la fonction modifier() et réaliser les tests fonctionnels.